

FORMATION EMBARQUÉE

TD 04 Vhdl

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 04 — VHDL : énoncés et corrigés détaillés

Tous les corrigés se simulent avec GHDL + GTKWave (ou edaplayground.com). Rappel de la discipline : **rien ne part en synthèse sans testbench.**

Exercice 1 — Multiplexeur 4 vers 1

Énoncé. Mux 4 → 1 (2 bits de sélection), en version concurrente puis en version process.

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;

entity mux4 is
    port (
        e0, e1, e2, e3 : in  std_logic;
        sel             : in  std_logic_vector(1 downto 0);
        s               : out std_logic
    );
end entity;

-- Version 1 : affectation sélectionnée (concurrente)
architecture concurrente of mux4 is
begin
    with sel select
        s <= e0 when "00",
            e1 when "01",
            e2 when "10",
            e3 when others; -- "others" couvre "11" ET les états 'X', 'U'...
end architecture;

-- Version 2 : process combinatoire
architecture avec_process of mux4 is
begin
    process (e0, e1, e2, e3, sel) -- TOUTES les entrées lues sont dans la liste
    begin
        case sel is
            when "00" => s <= e0;
            when "01" => s <= e1;
            when "10" => s <= e2;
            when others => s <= e3; -- chaque chemin affecte s → pas de latch
        end case;
    end process;
end architecture;
```

Points de correction : - `when others` est **obligatoire** avec `std_logic_vector` : le type a 9 valeurs possibles par bit (pas seulement 0/1), le `case` doit être exhaustif. - Liste de sensibilité complète + `s`

affecté dans toutes les branches : les deux conditions pour que la version process décrive bien du combinatoire pur (aucune bascule, aucun latch). - Les deux architectures se synthétisent **en exactement le même circuit**.

Testbench (exhaustif — 4 entrées, on peut tout tester) :

```
entity tb_mux4 is end entity;

architecture sim of tb_mux4 is
    signal e0, e1, e2, e3, s : std_logic;
    signal sel : std_logic_vector(1 downto 0);
begin
    dut : entity work.mux4(concurrente)
        port map (e0, e1, e2, e3, sel, s);

    stim : process
    begin
        (e0, e1, e2, e3) <= std_logic_vector("1010");
        sel <= "00"; wait for 10 ns;
        assert s = '1' report "sel=00 : attendu e0=1" severity error;
        sel <= "01"; wait for 10 ns;
        assert s = '0' report "sel=01 : attendu e1=0" severity error;
        sel <= "10"; wait for 10 ns;
        assert s = '1' report "sel=10 : attendu e2=1" severity error;
        sel <= "11"; wait for 10 ns;
        assert s = '0' report "sel=11 : attendu e3=0" severity error;
        report "mux4 : tous les tests passent";
        wait;
    end process;
end architecture;
```

Exercice 2 — Compteur BCD 0-9 + décodeur 7 segments

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity compteur_bcd is
    port (
        clk, reset, en : in std_logic;
        chiffre         : out unsigned(3 downto 0);
        seg             : out std_logic_vector(6 downto 0) -- gfedcba, actif bas
    );
end entity;

architecture rtl of compteur_bcd is
    signal cpt : unsigned(3 downto 0) := (others => '0');
begin
    -- Partie SÉQUENTIELLE : le compteur (des bascules)
    process (clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                cpt <= (others => '0');
            elsif en = '1' then
                if cpt = 9 then -- retour à 0 après 9 : BCD, pas binaire !
                    cpt <= (others => '0');
                else
                    cpt <= cpt + 1;
                end if;
            end if;
        end if;
    end process;

    chiffre <= cpt;

    -- Partie COMBINATOIRE : le décodeur 7 segments (une simple table)
    with cpt select
        seg <= "1000000" when "0000", -- 0
              "1111001" when "0001", -- 1
              "0100100" when "0010", -- 2
              "0110000" when "0011", -- 3
              "0011001" when "0100", -- 4
              "0010010" when "0101", -- 5
              "0000010" when "0110", -- 6
              "1111000" when "0111", -- 7
              "0000000" when "1000", -- 8
              "0010000" when "1001", -- 9
              "1111111" when others; -- éteint si valeur invalide
end architecture;
```

Extrait de testbench avec horloge générée et vérification du retour à zéro :

```

architecture sim of tb_compteur_bcd is
    signal clk, reset, en : std_logic := '0';
    signal chiffre : unsigned(3 downto 0);
    signal seg : std_logic_vector(6 downto 0);
    constant PERIODE : time := 10 ns;
begin
    dut : entity work.compteur_bcd port map (clk, reset, en, chiffre, seg);

    clk <= not clk after PERIODE / 2;    -- horloge perpétuelle en une ligne

    stim : process
    begin
        reset <= '1'; wait for 2 * PERIODE;
        reset <= '0'; en <= '1';
        for i in 0 to 9 loop
            assert chiffre = i
                report "attendu " & integer'image(i) severity error;
            wait for PERIODE;
        end loop;
        wait for PERIODE / 4;            -- s'écarter du front avant de lire
        assert chiffre = 0 report "pas revenu a 0 apres 9 !" severity error;
        report "compteur_bcd : OK";
        wait;
    end process;
end architecture;

```

Points de correction : séparation nette séquentiel (compteur) / combinatoire (décodeur) ; le test du passage 9 → 0 est LE cas intéressant — un testbench qui ne teste que 0 → 3 ne vaut rien.

Exercice 3 — Anti-rebond matériel (synchroniseur + filtre 20 ms)

Corrigé

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity debounce is
  generic (
    F_CLK      : natural := 50_000_000;
    DUREE_MS   : natural := 20
  );
  port (
    clk        : in  std_logic;
    btn_brut   : in  std_logic;    -- asynchrone, rebondissant
    btn_net    : out std_logic;    -- propre, synchrone
    front      : out std_logic     -- '1' pendant 1 cycle sur front montant net
  );
end entity;

architecture rtl of debounce is
  constant CYCLES_STABLES : natural := F_CLK / 1000 * DUREE_MS;
  signal sync0, sync1 : std_logic := '0';    -- synchroniseur 2 étages
  signal etat_net     : std_logic := '0';
  signal etat_prec    : std_logic := '0';
  signal cpt          : natural range 0 to CYCLES_STABLES := 0;
begin
  process (clk)
  begin
    if rising_edge(clk) then
      -- 1) Synchronisation : JAMAIS d'entrée asynchrone directe
      sync0 <= btn_brut;
      sync1 <= sync0;

      -- 2) Filtre : l'état doit rester stable CYCLES_STABLES cycles
      if sync1 /= etat_net then
        if cpt = CYCLES_STABLES - 1 then
          etat_net <= sync1;    -- état confirmé
          cpt <= 0;
        else
          cpt <= cpt + 1;
        end if;
      else
        cpt <= 0;              -- retombé pareil : on repart de zéro
      end if;

      -- 3) Mémoire pour le détecteur de front
      etat_prec <= etat_net;
    end if;
  end process;

  btn_net <= etat_net;
```

```
front    <= etat_net and not etat_prec;    -- exactement 1 cycle  
end architecture;
```

Points de correction : - Les **2 bascules de synchronisation** d'abord : sans elles, la métastabilité peut corrompre tout le circuit aval. C'est non négociable pour toute entrée externe. - Le compteur repart de zéro à **chaque** instabilité : 20 ms *consécutives* sont exigées. - Astuce testbench : passer `DUREE_MS => 1` et `F_CLK => 1000` en `generic map` pour simuler vite — c'est exactement à ça que servent les generics.

Exercice 4 — Feu tricolore avec bouton piéton

Corrigé (structure — réutilise le pattern du cours §4.3)

```
architecture rtl of feu_pieton is
    type t_etat is (VERT, ORANGE, ROUGE);
    signal etat : t_etat := ROUGE;
    signal cpt : unsigned(27 downto 0) := (others => '0');
    signal demande : std_logic := '0';
begin
    process (clk)
    begin
        if rising_edge(clk) then
            -- Mémorisation de la demande (front déjà nettoyé par debounce)
            if btn_front = '1' then
                demande <= '1';
            end if;

            cpt <= cpt + 1;
            case etat is
                when VERT =>
                    -- fin normale OU demande piéton après le délai minimal
                    if cpt = DUREE_VERT
                        or (demande = '1' and cpt >= DUREE_VERT_MINI) then
                        etat <= ORANGE;
                        cpt <= (others => '0');
                    end if;
                when ORANGE =>
                    if cpt = DUREE_ORANGE then
                        etat <= ROUGE;
                        cpt <= (others => '0');
                        demande <= '0'; -- demande servie
                    end if;
                when ROUGE =>
                    if cpt = DUREE_ROUGE then
                        etat <= VERT;
                        cpt <= (others => '0');
                    end if;
            end case;
        end if;
    end process;

    feu_vert <= '1' when etat = VERT else '0';
    feu_orange <= '1' when etat = ORANGE else '0';
    feu_rouge <= '1' when etat = ROUGE else '0';
end architecture;
```

À remarquer : c'est le même algorithme que le TD 01 en C et le TD 02 en C++ — drapeau mémorisé, durée minimale de vert, remise à zéro à la desserte. Seul le « moteur » change : ici, l'évaluation a lieu à chaque front d'horloge, en matériel. Si tu as vu cette correspondance tout seul, le paradigme est acquis.

Exercice 5 — Émetteur UART 115200 bauds

Corrigé complet

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity uart_tx is
    generic (
        F_CLK : natural := 50_000_000;
        BAUD  : natural := 115_200
    );
    port (
        clk      : in  std_logic;
        tx_start : in  std_logic;           -- impulsion 1 cycle
        tx_data  : in  std_logic_vector(7 downto 0);
        tx       : out std_logic;           -- la ligne série
        busy     : out std_logic
    );
end entity;

architecture rtl of uart_tx is
    constant TICKS_PAR_BIT : natural := F_CLK / BAUD; -- 50e6/115200 ≈ 434
    type t_etat is (REPOS, START, DONNEES, STOP);
    signal etat      : t_etat := REPOS;
    signal cpt_tick  : natural range 0 to TICKS_PAR_BIT - 1 := 0;
    signal idx_bit   : natural range 0 to 7 := 0;
    signal registre  : std_logic_vector(7 downto 0) := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            case etat is
                when REPOS =>
                    tx      <= '1';           -- ligne au repos = état haut
                    busy    <= '0';
                    if tx_start = '1' then
                        registre <= tx_data;   -- CAPTURER la donnée maintenant :
                        cpt_tick <= 0;         -- l'appelant peut la changer après
                        busy     <= '1';
                        etat     <= START;
                    end if;

                when START =>
                    tx <= '0';               -- bit de start
                    if cpt_tick = TICKS_PAR_BIT - 1 then
                        cpt_tick <= 0;
                        idx_bit  <= 0;
                        etat     <= DONNEES;
                    else
                        cpt_tick <= cpt_tick + 1;
                    end if;
            end case;
        end if;
    end process;
end architecture;
```

```

when DONNEES =>
    tx <= registre(idx_bit);    -- LSB en premier (norme UART)
    if cpt_tick = TICKS_PAR_BIT - 1 then
        cpt_tick <= 0;
        if idx_bit = 7 then
            etat <= STOP;
        else
            idx_bit <= idx_bit + 1;
        end if;
    else
        cpt_tick <= cpt_tick + 1;
    end if;

when STOP =>
    tx <= '1';                -- bit de stop
    if cpt_tick = TICKS_PAR_BIT - 1 then
        etat <= REPOS;
    else
        cpt_tick <= cpt_tick + 1;
    end if;

end case;
end if;
end process;
end architecture;

```

Testbench (envoi de 0x55 = **01010101** , motif idéal : il alterne) :

```

architecture sim of tb_uart_tx is
    signal clk, tx_start, tx, busy : std_logic := '0';
    signal tx_data : std_logic_vector(7 downto 0);
    -- Astuce : generics réduits pour une simulation courte et lisible
    constant F : natural := 10 * 115200;    -- 10 ticks par bit seulement
begin
    dut : entity work._uart_tx
        generic map (F_CLK => F, BAUD => 115_200)
        port map (clk, tx_start, tx_data, tx, busy);

    clk <= not clk after 50 ns;

    stim : process
    begin
        wait for 200 ns;
        tx_data <= x"55";
        tx_start <= '1'; wait for 100 ns; tx_start <= '0';    -- impulsion
        wait until busy = '0';                                -- fin d'émission
        report "trame emise, verifier le chronogramme dans GTKWave";
        wait;
    end process;
end architecture;

```

Ce qu'on vérifie au chronogramme : ligne haute au repos → start bas pendant 1 temps bit →

1,0,1,0,1,0,1,0 (0x55, LSB d'abord) → stop haut, chaque bit durant exactement **TICKS_PAR_BIT** cycles.

Points de correction : la capture de **tx_data** dans **registre** à l'acceptation (l'oublier = donnée

corrompue si l'appelant la change en cours d'émission), le LSB en premier, et `busy` qui couvre toute la trame.

Auto-évaluation avant le TP 2

Sans notes : écrire un process synchrone canonique ; expliquer latch involontaire et comment l'éviter ; justifier le synchroniseur 2 bascules ; dessiner la trame UART 8N1 ; expliquer pourquoi `wait for` est interdit en synthèse.

 Passe au [TP 2 — UART complet en VHDL](#).